

The Setup

The test simulates what happens when an attacker already has one foot inside the door. No phishing, no firewall bypass needed. The starting point is a single low-privileged domain account: `hporter` with the password `Gr8hambino!.` The domain is `INLANEFREIGHT.LOCAL`, and the goal is to see how far that one account can go.

This approach is called an **assumed-breach** model. The idea is that perimeter defenses are considered already defeated, and the question becomes: what can an insider or a compromised account actually do?

Stage 1 — Confirming the Foothold

Before anything else, the team verifies that `hporter`'s credentials actually work. The tool used is **NetExec** (abbreviated as `nxc`), which sends an authentication attempt over **SMB** (Server Message Block, the file-sharing protocol Windows uses internally). `DC01` at `172.16.8.3` responds successfully. The account is valid and active on the domain.

Why NetExec over alternatives like CrackMapExec? NetExec is the actively maintained fork; CrackMapExec has been largely abandoned. Same functionality, but NetExec gets security patches and updates.

Stage 2 — Mapping the Domain with BloodHound

With confirmed credentials, the next move is enumeration — figuring out what the domain looks like from the inside. **BloodHound** (specifically `bloodhound-python`) connects to the domain controller over **LDAP** and pulls down every object it can reach: users, groups, computers, and most importantly, **ACLs** (Access Control Lists).

ACLs in Active Directory define who has permission to do what to whom. Most administrators focus on group memberships, but the real attack surface is often in these ACL entries, which are rarely audited.

BloodHound renders all of this as a graph. The team runs a shortest-path query and finds something immediately useful: `hporter` has a permission called **ForceChangePassword** over another account, `ssmallis`.

Stage 3 — Abusing ForceChangePassword

ForceChangePassword is exactly what it sounds like. It allows one account to reset another user's password without knowing the current one. This is a legitimate AD delegation intended for helpdesk staff, but it was misconfigured here on a regular user account.

The tool used to exploit this is **bloodyAD**, which sends a password reset request directly over LDAP. The team resets `ssmall`'s password to `mazoniakid`. No interaction with the actual user. No alert triggered. Ownership of that account now belongs to the attacker.

Why bloodyAD over PowerView? PowerView requires execution on a Windows host. bloodyAD runs natively from the Linux attack machine over the network, which keeps the attacker's footprint smaller.

Stage 4 — Harvesting Credentials from a File Share

Now operating as `ssmall`, the team checks what file shares are accessible on DC01. NetExec's `spider_plus` module crawls the "Department Shares" share and finds a backup script sitting in an IT folder.

The script contains hardcoded credentials in plaintext: username `backupadm`, password `!qazXSW@`. Backup scripts frequently carry credentials because they need to authenticate automatically. The problem is leaving them readable to all domain users.

Stage 5 — Lateral Movement to MS01 via WinRM

With `backupadm`'s credentials, the team checks whether that account has remote access anywhere. **WinRM** (Windows Remote Management) is the protocol Windows uses for remote PowerShell sessions, running on port 5985.

NetExec confirms that `backupadm` can authenticate to MS01 at `172.16.8.50`. The team connects using **evil-winrm**, which gives a full interactive PowerShell shell on the server. Lateral movement complete — the attacker is now inside MS01 as a service account.

Stage 6 — Finding Credentials in an Unattend.xml File

On MS01, the team checks a well-known location: `C:\Panther\unattend.xml`. This is a deployment answer file, left behind by Windows setup processes. When IT builds a machine using automated deployment, the setup credentials are written into this file so the system can configure itself without human input.

The file was never cleaned up. It contains the local administrator credentials for MS01 in plaintext: `ilfserveradm/Sys26Admin`. This is a common real-world find; deployment teams rarely have a checklist item for removing these after provisioning.

Stage 7 — Local Privilege Escalation via Sysax

Currently, the team is on MS01 as `backupadm`, which is not a local administrator. To go further, they need SYSTEM-level access on the machine. Using the `ilfserveradm` credentials recovered from `unattend.xml`, they RDP into the machine with **xfreerdp** for a graphical session.

On the machine, they find a third-party application called **Sysax Automation**, which is a scheduled task and file transfer service. The service runs as NT AUTHORITY\SYSTEM and allows users to configure custom batch scripts as triggers.

The attacker creates a batch script that adds `ilfserveradm` to the local Administrators group, then fires it through the Sysax interface. The service executes it as SYSTEM. Local privilege escalation achieved without touching any Windows kernel exploits.

Stage 8 — Dumping LSA Secrets to Get mssqladm

Now as a local administrator on MS01, the team runs **Mimikatz** with elevated privileges. The specific command is `lsadump::secrets`, which reads the **LSA Secrets** store in the Windows registry.

LSA Secrets is where Windows caches certain credentials, most commonly for services and applications configured to use **Autologon** — where the machine logs in automatically at boot using a stored account. The registry key `HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Winlogon` confirms the stored username is `mssqladm`.

Mimikatz extracts the plaintext password for `mssqladm`: `DBAilfreight1!`. This account is a domain user with specific permissions, not just a local one.

Stage 9 — Targeted Kerberoasting via GenericWrite

BloodHound's earlier enumeration showed that `mssqladm` has **GenericWrite** permission over a user named `ttimmons`. GenericWrite allows modification of most attributes on an AD object.

The attack here is called **targeted Kerberoasting**. Kerberoasting normally works on accounts that already have a Service Principal Name (SPN) registered, since those accounts' tickets are

encrypted with their password and can be cracked offline. `ttimmons` doesn't have an SPN. But with `GenericWrite`, the attacker can *add* one.

Using `bloodyAD`, they inject a fake SPN (`acmetesting/LEGIT`) onto `ttimmons`'s account. Then they use **`GetUserSPNs.py`** from the Impacket suite to request a Kerberos TGS (Ticket Granting Service) ticket for that account. The ticket is encrypted with `ttimmons`'s password hash.

That ticket is saved to a file and fed to **Hashcat** for offline cracking. Hashcat runs through the `rockyou.txt` wordlist against the RC4-HMAC (etype 23) ticket hash and recovers the password: `Repeat09`. Hashcat over John the Ripper because Hashcat uses GPU acceleration, which is orders of magnitude faster on this hash type.

Stage 10 — Adding `ttimmons` to Server Admins

BloodHound shows that `ttimmons` has **GenericAll** over a group called "Server Admins." `GenericAll` is full control — read, write, modify membership, everything.

The team uses `bloodyAD` again to add `ttimmons` directly to the Server Admins group. No administrator approval needed. The change happens at the LDAP level and is reflected immediately in AD.

Stage 11 — DCSync: Dumping All Domain Hashes

This is the end game. BloodHound shows that the Server Admins group holds two extended rights on the domain root object: **DS-Replication-Get-Changes** and **DS-Replication-Get-Changes-All**. Together, these rights allow an account to replicate the entire AD database, specifically the `NTDS.DIT`, which contains every user's password hash.

This is the **DCSync** attack. Normally, only Domain Controllers replicate this data between each other. But because Server Admins has those rights, `ttimmons` can impersonate a Domain Controller and request the same replication.

The tool is **`secretsdump.py`** from the Impacket suite. It connects to `DC01` over the network — no agent, no code execution on the DC — and requests a replication dump. Every account's NTLM hash comes back, including the built-in Administrator's:

```
fd1f7e5564060258ea787dabb6e6afa2.
```

Why `secretsdump.py` here instead of `Mimikatz`? `Mimikatz` requires code to run on the target DC, which is noisy and risky. `secretsdump.py` performs the entire operation remotely using only legitimate AD replication protocols.

Stage 12 — Pass-the-Hash as Domain Administrator

The team never cracks the Administrator's hash. They don't need to. **Pass-the-Hash** is a technique where the raw NTLM hash is used directly for authentication, because Windows's NTLM authentication protocol accepts it in place of the actual password.

Using `evil-winrm` with the `-H` flag, they authenticate to DC01 as Administrator using that hash. Full domain controller access. The engagement is complete.

What This All Means

The entire chain from `hporter` to Domain Administrator involved zero zero-day exploits. Every step used either native Windows functionality or permissions that someone had deliberately (if carelessly) configured in Active Directory. The report's key argument to the lecturer is this: the environment was compromised not because of a single catastrophic bug, but because ten small, unmonitored configuration decisions created an unbroken path from a regular user account to complete domain control.

The remediation section follows that same logic — fix the chain at any link, and the entire attack path breaks. That is what the prioritized roadmap in Section 6 addresses.